

Last updated on **Mar 23, 2026**

Get started with data warehousing using Databricks SQL

If you're a data analyst who works primarily with SQL queries and your favorite BI tools, Databricks SQL provides an intuitive environment for running ad-hoc queries and creating dashboards on data stored in your data lake.

Learn the basics

Step	Description
Basic Databricks SQL concepts	Familiarize yourself with the core interfaces, query concepts, data management tools, and authentication model in Databricks SQL.
Data warehousing architecture	Understand lakehouse architecture, medallion layers, and data modeling approaches for building a data warehouse.

Build your first assets

Step	Description
Explore sample dashboards	Import and interact with dashboards from the Dashboard Samples Gallery that visualize queries on sample data.
 Ask Genie	

Step	Description
Create an AI/BI dashboard	Build a dashboard to explore data and share insights with your team using AI-assisted authoring and an enhanced visualization library.
Create a SQL warehouse	Provision a SQL warehouse to start running queries on your data.

Connect and automate

Step	Description
Use Databricks SQL with a notebook	Attach a notebook to a SQL warehouse to run SQL alongside Python, Scala, or R.
Load data with COPY INTO	Use COPY INTO to ingest data from cloud object storage into your Databricks SQL tables.
Use Databricks SQL in a job	Create, schedule, and monitor workflows that include Databricks SQL queries, dashboards, and alerts.
Connect BI tools with Partner Connect	Connect your Databricks workspace to a BI and visualization partner solution.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

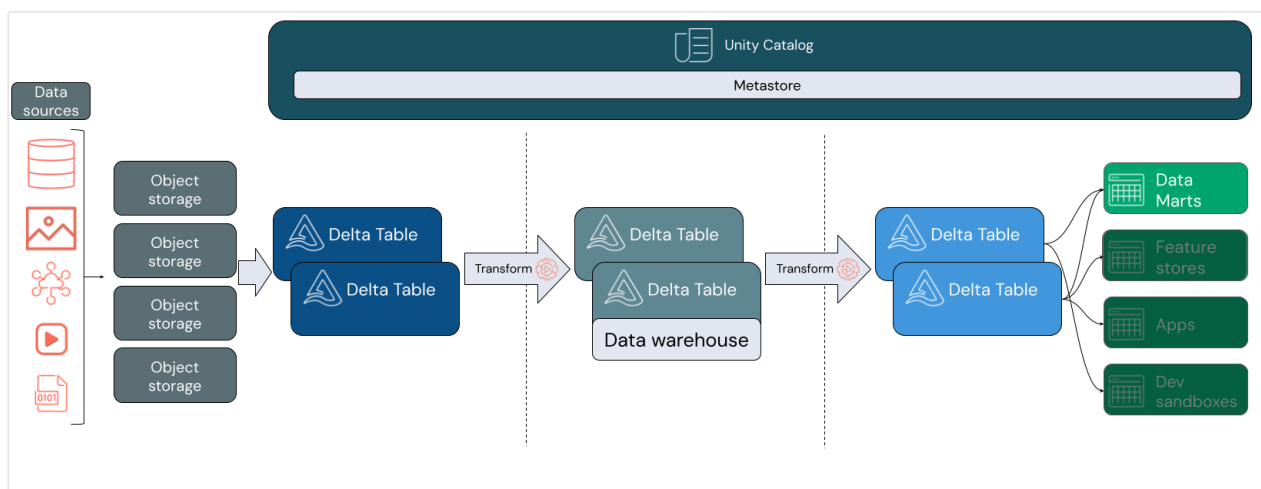
Last updated on **May 7, 2026**

Data warehousing architecture

Data warehousing refers to collecting and storing data from multiple sources so it can be quickly accessed for business insights and reporting. This page contains key concepts for building a data warehouse in your data lakehouse.

Data warehousing in your lakehouse

The lakehouse architecture and Databricks SQL bring cloud data warehousing capabilities to your data lakes. Using familiar data structures, relations, and management tools, you can model a highly-performant, cost-effective data warehouse that runs directly on your data lake. For more information, see [What is a lakehouse?](#)



As with a traditional data warehouse, you model data according to business requirements and then serve it to your end users for analytics and reports. Unlike a traditional data warehouse, you can avoid siloing your business analytics data or creating redundant copies that quickly become stale.

Building a data warehouse inside your lakehouse allows you to bring all your data into a single system and take advantage of features such as Unity Catalog and

[Unity Catalog](#) adds a unified governance model so that you can secure and audit data access and provide lineage information on downstream tables. [Delta Lake](#) adds ACID transactions and schema evolution, among other powerful tools for keeping your data reliable, scalable, and high-quality.

What is Databricks SQL?

Databricks SQL is the collection of services that bring data warehousing capabilities and performance to your existing data lakes. Databricks SQL supports open formats and standard ANSI SQL. An in-platform SQL editor and dashboarding tools allow team members to collaborate with other Databricks users directly in the workspace.

Databricks SQL also integrates with a variety of tools so that analysts can author queries and dashboards in their favorite environments without adjusting to a new platform.

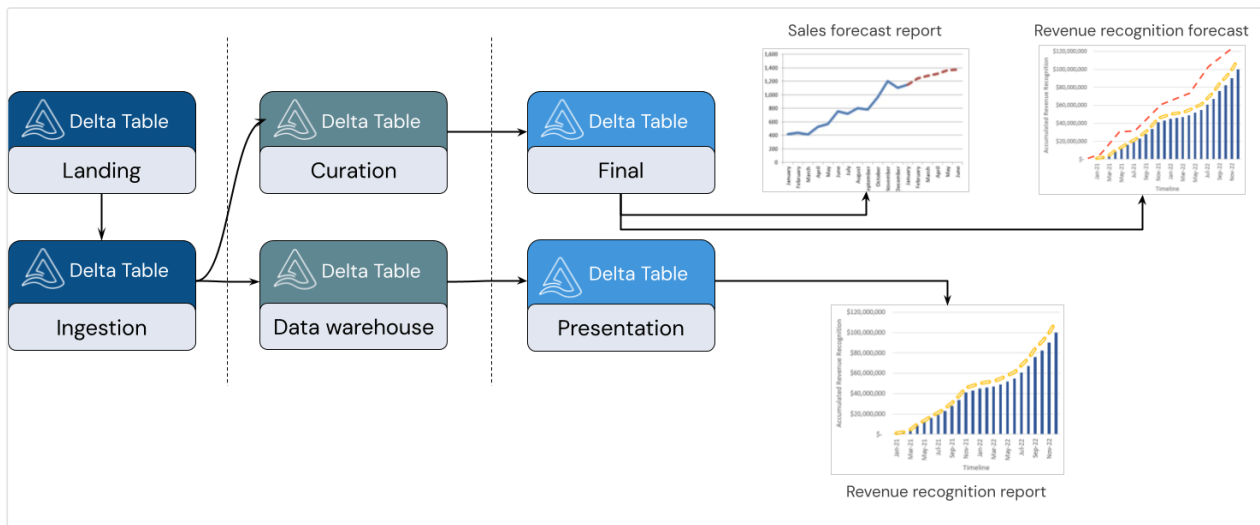
Databricks SQL provides general compute resources that are executed against the tables in the lakehouse. Databricks SQL is powered by [SQL warehouses](#), formerly called SQL endpoints, offering scalable SQL compute resources decoupled from storage.

See [SQL warehouses](#) for more information on SQL Warehouse defaults and options.

Databricks SQL integrates with Unity Catalog, allowing you to discover, audit, and govern data assets from one place. To learn more, see [What is Unity Catalog?](#)

Data modeling on Databricks

A lakehouse supports a variety of modeling styles. The following image shows how data is curated and modeled as it moves through different layers of a lakehouse.



Medallion architecture

The medallion architecture is a data design pattern that describes a series of incrementally refined data layers that provide a basic structure in the lakehouse. The bronze, silver, and gold layers signify increasing levels of data quality, with gold representing the highest quality. For more information, see [What is the medallion lakehouse architecture?](#)

Inside a lakehouse, each layer can contain one or more tables. The data warehouse is modeled in the silver layer and feeds specialized data marts in the gold layer.

Bronze layer

Data can enter your lakehouse in any format and through any combination of batch or streaming transactions. The bronze layer provides the landing space for all of your raw data in its original format. That data is converted to Delta tables.

Silver layer

The silver layer brings the data from different sources together. For the part of the business that focuses on data science and machine learning applications, this is where you start to curate meaningful data assets. This process is often marked by a focus on speed and agility.

The silver layer is also where you can carefully integrate data from disparate sources to build a data warehouse in alignment with your existing business processes. Often,

this data follows a Third Normal Form (3NF) or Data Vault model. Specifying primary and foreign key constraints allows end users to understand table relationships when using Unity Catalog. Your data warehouse should serve as the single source of truth for your data marts.

The data warehouse itself is schema-on-write and atomic. It is optimized for change, so you can quickly modify the data warehouse to match your current needs when your business processes change or evolve.

Gold layer

The gold layer is the presentation layer, which can contain one or more data marts. Frequently, data marts are dimensional models in the form of a set of related tables that capture a specific business perspective.

The gold layer also houses departmental and data science sandboxes to enable self-service analytics and data science across the enterprise. Providing these sandboxes and their own separate compute clusters prevents the Business teams from creating copies of data outside the lakehouse.

Next steps

To learn more about the principles and best practices for implementing and operating a lakehouse using **Databricks**, see [Lakehouse architecture](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

 Ask Genie

Last updated on **Apr 27, 2026**

Databricks SQL concepts

This page introduces the core concepts you need to use Databricks SQL effectively.

Interfaces and tools

Interface	Description
SQL editor	A browser-based interface for writing and running SQL queries. Includes integrated AI assistance, code comments, and version history.
AI/BI	A business intelligence solution that includes dashboards and Genie Spaces for self-service data analysis and conversational data exploration.
Notebooks	Interactive documents that combine SQL with Python, Scala, or R. Attach a notebook to a SQL warehouse to run SQL alongside other languages.
Metric views	A semantic layer for defining reusable business metrics with consistent calculations across queries and dashboards.
Alerts	Automated monitors that evaluate query results against a condition and deliver notifications when the condition is met.
Jobs	Scheduled execution of SQL queries for automated data processing and reporting workflows.
ETL	Define and refresh streaming tables and materialized views in Databricks SQL for incremental ETL pipelines.

 Ask Genie

Interface	Description
REST API	A programmatic interface for automating and managing Databricks SQL objects.

Queries and computation

Concept	Description
Query	A SQL statement that retrieves or transforms data. Queries in Databricks SQL are saved, versioned, and reusable.
SQL warehouse	The compute resource that executes SQL queries. All Databricks SQL interfaces run queries on a SQL warehouse.
Query history	A record of executed queries including execution time, resource usage, and performance characteristics.
Query profile	A detailed view of a query's execution plan. Use it to identify bottlenecks and optimization opportunities.
Visualization	A graphical representation of query results, such as a bar chart, line chart, or table.

Data management

Concept	Description
Dashboard	A collection of visualizations and commentary built from one or more queries. Dashboards are shareable and support interactive filtering.
 Ask Genie	

Concept	Description
Metric view	A named, reusable definition of a business metric backed by a SQL expression. Enforces consistent calculations across teams and tools.
Streaming table	A table that ingests data incrementally from a streaming source, managed by a Databricks SQL pipeline.
Materialized view	A precomputed view whose results are stored and refreshed automatically by a Databricks SQL pipeline.

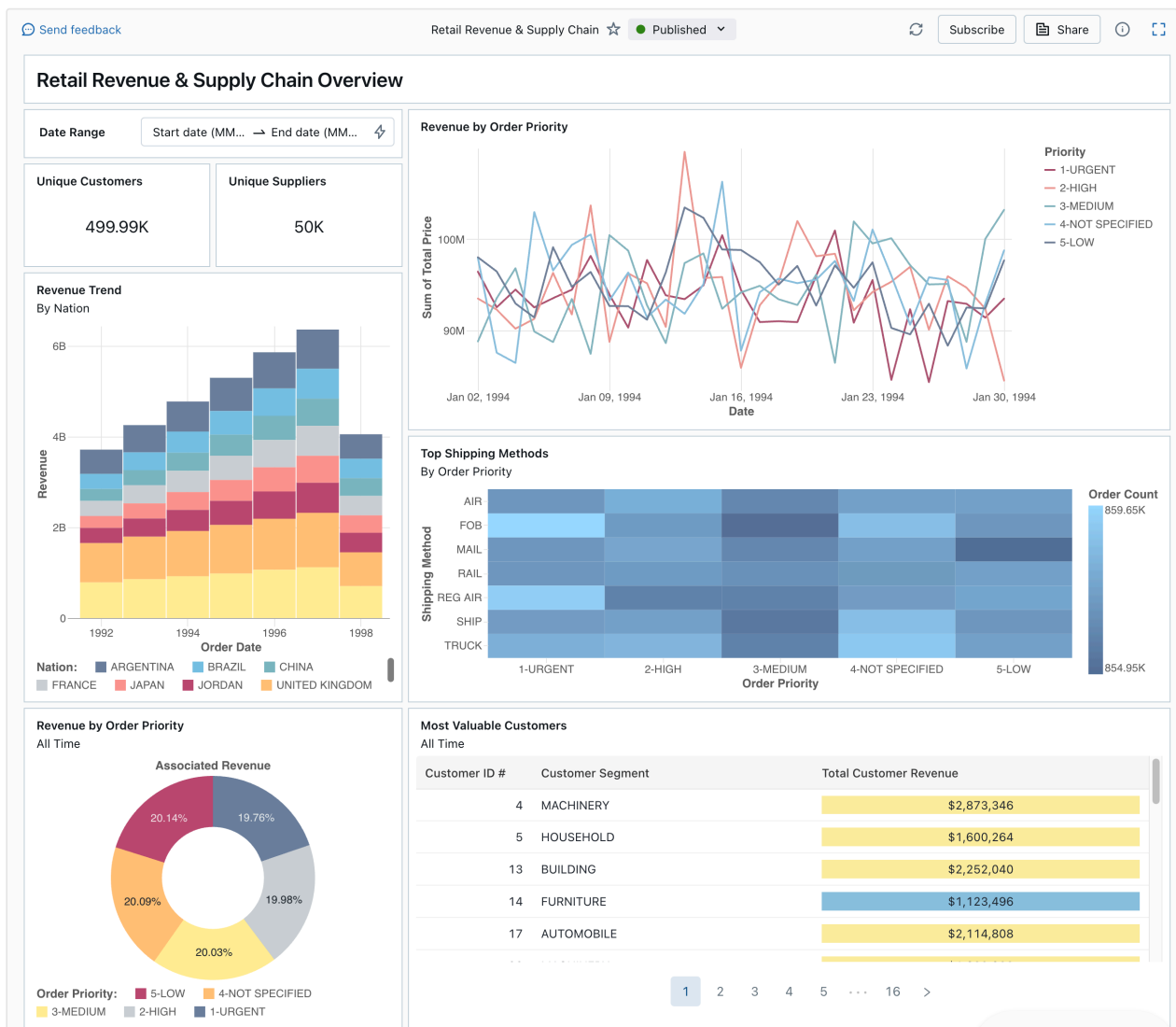
Authentication and authorization

Concept	Description
User and group	A user is a unique individual with access to the Databricks workspace. A group is a collection of users managed together for access control purposes.
Personal access token	A credential used to authenticate to the REST API and to connect third-party tools to SQL warehouses.
Access control list	A set of permissions attached to an object that controls which principals can perform which actions on it.
Unity Catalog	The governance layer for Databricks. Provides centralized access control, auditing, lineage, and data discovery across workspaces.

Last updated on **Apr 29, 2026**

Tutorial: Use sample dashboards

This tutorial shows you how to import and use sample dashboards from the samples gallery. These dashboards illustrate some of the rich visualizations you can use to gain insights from your data. No setup is required. These dashboards use data already available in your workspace and rely on a compute resource (called a SQL warehouse) already configured. You don't need to be an administrator to get started.



See [Dashboards](#) to learn about all of the visualization types and features available for dashboards.

Import a dashboard

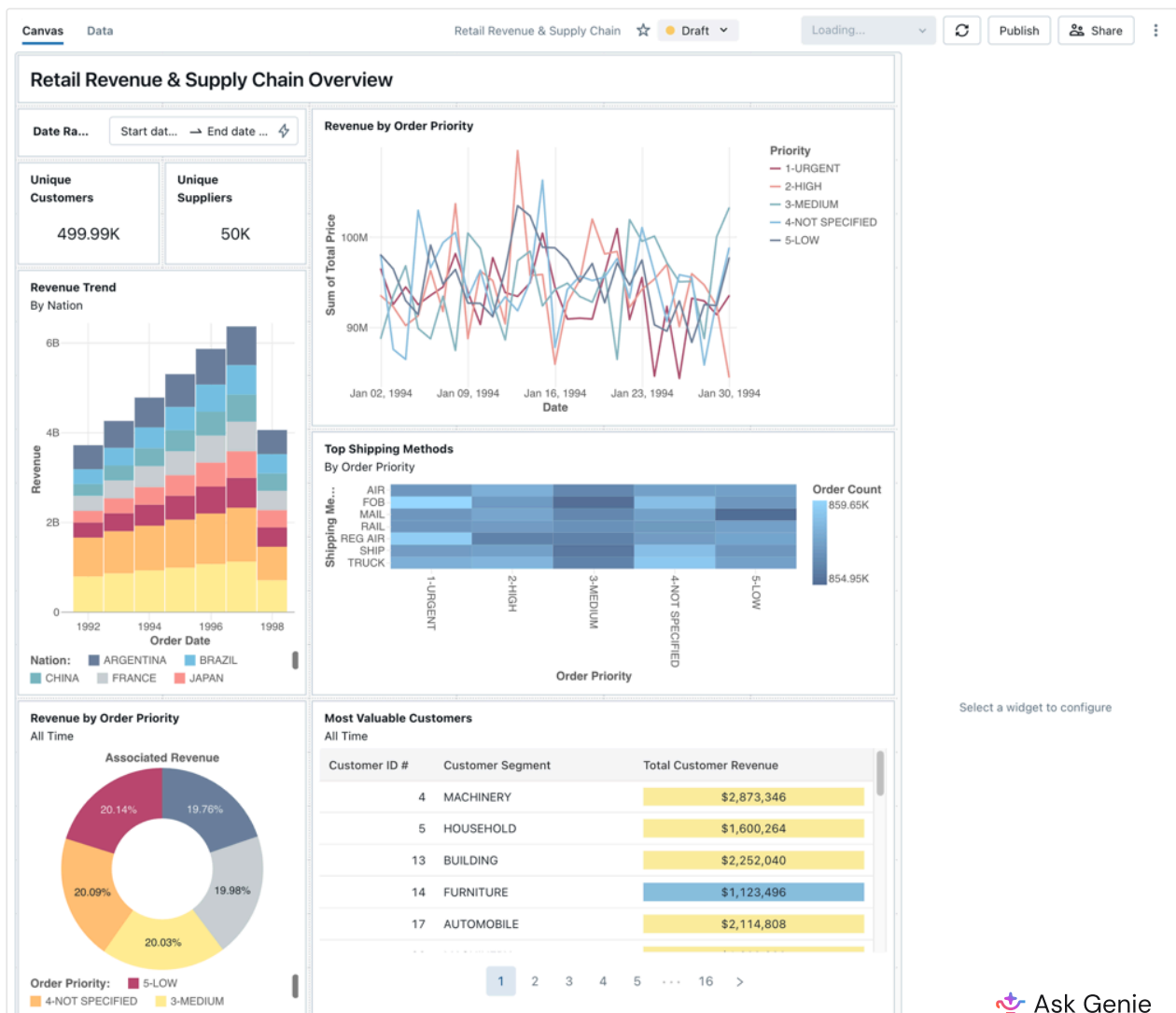
1. In the sidebar, click  **Dashboards**

If your workspace has any saved dashboards, they are listed.

2. Click **View samples gallery**.

3. In the **Retail Revenue & Supply Chain** tile, click **Import**. The dashboard is imported into your workspace, and you are the owner.

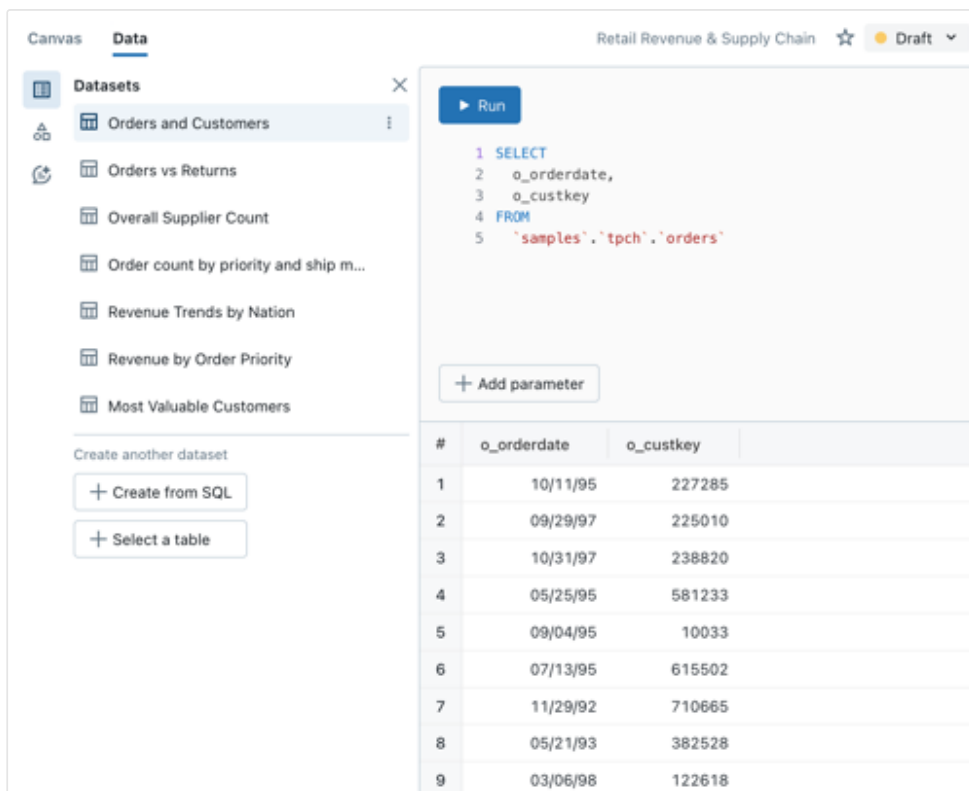
The imported draft dashboard appears, and its visualizations are refreshed.



You can import a sample dashboard multiple times, and multiple users can each import it. You can also import the **NYC Taxi Trip Analysis** dashboard.

Explore a visualization's query

1. Each visualization in a dashboard is the result of a query. You can access all queries in the **Data** tab on the draft dashboard. Click **Data** in the upper-left corner of the screen. Then, click the dataset you want to view to see the associated query.



The screenshot shows the 'Data' tab of a dashboard editor. On the left, a list of datasets is displayed, including 'Orders and Customers', 'Orders vs Returns', 'Overall Supplier Count', 'Order count by priority and ship m...', 'Revenue Trends by Nation', 'Revenue by Order Priority', and 'Most Valuable Customers'. The 'Orders and Customers' dataset is selected. To the right, the SQL query editor is open, showing a query that selects 'o_orderdate' and 'o_custkey' from the 'orders' table in the 'samples' catalog. Below the query, a table displays the results of the query, with columns for '#', 'o_orderdate', and 'o_custkey'.

#	o_orderdate	o_custkey
1	10/11/95	227285
2	09/29/97	225010
3	10/31/97	238820
4	05/25/95	581233
5	09/04/95	10033
6	07/13/95	615502
7	11/29/92	710665
8	05/21/93	382528
9	03/06/98	122618

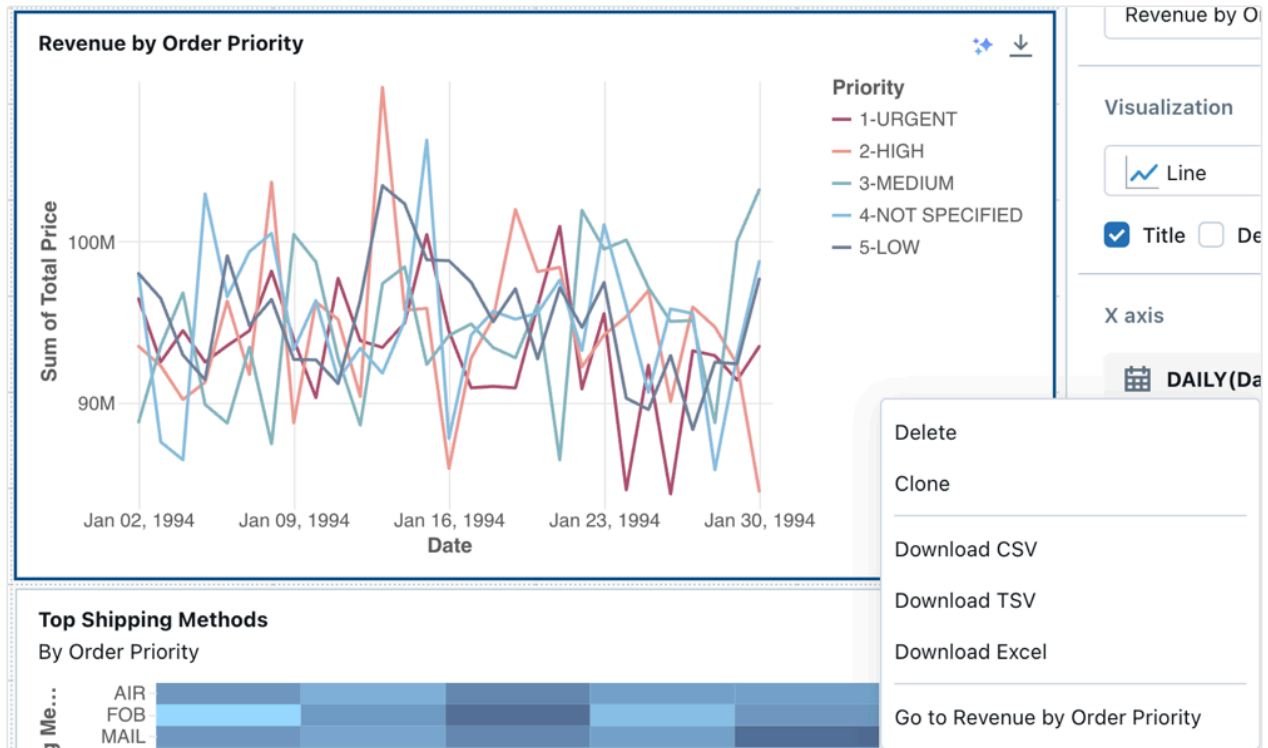
The SQL editor includes the query and results, which are shown in a table below the query.

The sample dashboards use data in the `samples` catalog, separate from data in your workspace. The `samples` catalog is available to every workspace but is read-only.

1. Click the **Canvas** tab to go back to the canvas that shows the dashboard's visualization widgets.

Interact with a visualization

1. Hover over the **Revenue by Order Priority** visualization.
2. Click each **Priority** in the legend to focus on that group of data and hide the other lines.
3. Right-click on the visualization to see its context menu. You can delete or clone a visualization. You can also download the associated dataset as a CSV, TSV, or Excel file. Click **Go to Revenue by Order Priority** to view the associated query.



The query opens on the **Data** tab of your dashboard.

Publish the dashboard

- Click **Publish** at the top of the page. A **Publish** dialog appears.
- Click **Publish** in the dialog to create a sharable, non-editable version of your dashboard. This dashboard is published with your credentials embedded by default. This means that other viewers use your credentials to access the data and compute to generate visualizations on your dashboard. See [Publish a dashboard](#).
- Use the switcher at the top of the page to view your published dashboard.

Draft

▼

✔ d...

Serverless

✔ Draft

Edit the layout of this dashboard

Published

Feb 26, 2024 03:52 PM

Share the dashboard

To share a dashboard with colleagues in your workspace:

1. Click **Share** at the top of the page.
2. Select a user or group in your workspace.

To share the dashboard with all users in the workspace, select **All workspace users**. Then, click **Add**.

3. Select the permission to grant.

Sharing: Retail Revenue & Supply Chain

Do you want information about permission levels? [Learn more](#)

All workspace users X

+

▼

Can Manage ▼

☐ Email new users (excluding groups)

Cancel

Add

[Copy link](#)

To share a dashboard with account users:

1. Under **Sharing settings** at the bottom of the sharing dialog, click **Anyone in my account can view**.

This means that anyone who is registered to your Databricks account can use a link to access your dashboard. If you have embedded your credentials, account-level users don't need workspace access to view your dashboard.

2. Close the form.

Schedule automatic dashboard refreshes

You can schedule the dashboard to refresh at an interval automatically.

1. At the top of the page, click **Schedule**.
2. Click **Add schedule**.
3. Select an interval, such as **Every 1 hour at 5 minutes past the hour**. The SQL warehouse that you selected to run your queries is used to run the dashboard's queries and generate visualizations when the dashboard is refreshed.

Workspace admin users can create, configure, and delete SQL warehouses.

4. Click **Create**.

The dialog shows all schedules associated with the dashboard.

5. Optionally, click **Subscribe** to add yourself as a subscriber and receive an email with a PDF snapshot of the dashboard after a scheduled run completes.

You can use the kebab menu  to edit the schedule and add more subscribers. See [Manage scheduled dashboard updates and subscriptions](#).

6. To delete an existing schedule for a dashboard:

- i. Click **Subscribe**.
- ii. Click the kebab menu  on the right.

iii. Click **Delete**.

Is this page

as this page helpful?

 Yes

 No

 Send feedback

Last updated on **Apr 10, 2026**

ETL in Databricks SQL

When dealing with large amounts of data, you need a pipeline that can process only the new and changed records instead of reprocessing the entire dataset. This is called incremental ETL. In Databricks SQL, you can build incremental ETL pipelines using streaming tables and materialized views, without writing procedural code or scheduling manual refreshes.

This tutorial walks you through a common pattern: tracking product changes over time. You create a source table, capture change events, build a dimension table that preserves the full history of each product, and add an aggregate reporting layer on top.

The key feature in this tutorial is `AUTO CDC`. In a traditional warehouse, you would write complex `MERGE INTO` statements to reconcile insert, update, and delete events into a target table. This approach is error-prone, especially when events arrive out of order. `AUTO CDC` handles this for you. You declare the business key, the sequencing column, and whether you want SCD Type 1 (latest value only) or SCD Type 2 (full history), and Databricks applies the correct merge logic automatically. For an overview of CDC, see [The AUTO CDC APIs: Simplify change data capture with pipelines](#).

By the end of this tutorial, you will have:

1. Created a source table that tracks changes with Change Data Feed.
2. Inspected the raw change data to understand the CDC event stream.
3. Used `AUTO CDC` to build an SCD Type 2 dimension table from those events.
4. Processed delete events incrementally through the pipeline.
5. Created a materialized view that incrementally maintains an aggregate report.
6. Configured `SCHEDULE REFRESH EVERY 1 DAY` so changes propagate automatically through the pipeline.

Requirements

To complete this tutorial, you must meet the following requirements:

- A Databricks workspace with [Unity Catalog](#) enabled.
- A [SQL warehouse](#) (serverless or pro).
- Have permission to [create a compute resource](#) or [access to a compute resource](#).
- [Serverless compute](#) enabled for your account. See [Features with limited regional availability](#).

Step 1: Set up your catalog and schema

Open the [Databricks SQL editor](#) and set your working catalog and schema. You must have permission to `USE` the catalog and schema you select:

SQL

```
USE CATALOG <your-catalog>;  
USE SCHEMA <your-schema>;
```

Step 2: Create a source table and load data

Create a `products` table with [Use Delta Lake change data feed on Databricks](#) (CDF) enabled. CDF is a Delta Lake feature that records every insert, update, and delete as a queryable change log. This is similar to a CDC stream from a transactional source system, except the changes are captured directly within the Delta table rather than from an external log. You use CDF here to generate the change events that the downstream pipeline will consume.

1. Create the table and load initial records:

SQL

 Ask Genie

```
CREATE OR REPLACE TABLE products (
  product_id INT,
  product_name STRING,
  category STRING,
  warehouse STRING
)
TBLPROPERTIES (delta.enableChangeDataFeed = true);

INSERT INTO products VALUES
(1, 'Spoon', 'Cutlery', 'Seattle'),
(2, 'Fork', 'Cutlery', 'Portland'),
(3, 'Knife', 'Cutlery', 'Denver'),
(4, 'Chair', 'Furniture', 'Austin'),
(5, 'Table', 'Furniture', 'Chicago'),
(6, 'Lamp', 'Lighting', 'Boston'),
(7, 'Mug', 'Kitchenware', 'Seattle'),
(8, 'Plate', 'Kitchenware', 'Atlanta'),
(9, 'Bowl', 'Kitchenware', 'Dallas'),
(10, 'Glass', 'Kitchenware', 'Phoenix');
```

2. Simulate upstream changes, including new products, a warehouse move, and a category reassignment:

SQL

```
INSERT INTO products VALUES
(11, 'Napkin', 'Dining', 'San Francisco'),
(12, 'Coaster', 'Dining', 'New York');

UPDATE products SET warehouse = 'Los Angeles' WHERE product_id = 1;
UPDATE products SET category = 'Dining' WHERE product_id = 2;
```

Step 3: Query the change data feed

Before building the downstream pipeline, it helps to look at the raw change events so you can understand what `AUTO CDC` will process. The `table_changes()` function reads the CDF log and returns every captured operation along with metadata columns:

SQL

```
SELECT
  product_id, product_name, warehouse,
  _change_type, _commit_version
FROM table_changes('products', 1)
ORDER BY _commit_version, product_id;
```

For example, the Spoon has three events: an `insert` (Seattle), an `update_preimage` (Seattle), and an `update_postimage` (Los Angeles).

Notice that a single logical change (for example, moving the Spoon to a different warehouse) produces multiple events: a preimage and a postimage. In a traditional warehouse, you would write a `MERGE` statement to reconcile all of these events into a target table, handling inserts, updates, and deletes with separate logic, and making sure events are applied in the correct order. This is exactly the complexity that `AUTO CDC` eliminates in the next step.

Step 4: Build an SCD Type 2 dimension with `AUTO CDC`

ⓘ BETA

`AUTO CDC` is in [Beta](#). Requires Databricks Runtime 17.3 or above.

A [streaming table](#) processes data incrementally. On each refresh, it reads only the new rows since the last run, so it does not need to reprocess the full dataset. This makes it well-suited for high-volume or frequently changing sources.

`AUTO CDC` adds change data capture processing on top of a streaming table. Instead of writing a `MERGE INTO` statement that manually handles inserts, updates, and deletes, you declare the business key and sequencing column and let Databricks apply the correct logic. `AUTO CDC` also handles out-of-order events automatically, which is a common problem when using `MERGE INTO` to handle events arriving from distributed systems or batch loads with overlapping timestamps.

The following statement creates an SCD Type 2 table that preserves the full version history of each product. Each version gets `__START_AT` and `__END_AT` timestamps. A `NULL` in `__END_AT` marks the current version.

 Ask Genie

SQL

```
CREATE OR REFRESH STREAMING TABLE products_history
SCHEDULE REFRESH EVERY 1 DAY
FLOW AUTO CDC
FROM STREAM products WITH (readChangeFeed = true)
KEYS (product_id)
APPLY AS DELETE WHEN _change_type = 'delete'
SEQUENCE BY _commit_timestamp
COLUMNS * EXCEPT (_change_type, _commit_version, _commit_timestamp)
STORED AS SCD TYPE 2;
```

- `SCHEDULE REFRESH EVERY 1 DAY`: refreshes the table on a daily schedule.
- `FLOW AUTO CDC`: declares this as a CDC flow. Databricks applies insert, update, and delete semantics automatically.
- `KEYS (product_id)`: the business key. Events with the same key are merged into versioned rows.
- `APPLY AS DELETE WHEN _change_type = 'delete'`: closes out the current version when a delete event arrives. This lets you define the condition that identifies a delete event.
- `SEQUENCE BY _commit_timestamp`: establishes event ordering. Handles out-of-order arrivals correctly.
- `STORED AS SCD TYPE 2`: retains full history. `AUTO CDC` supports both SCD Type 1 and SCD Type 2.

Query the dimension table:

SQL

```
SELECT product_id, product_name, warehouse, __START_AT, __END_AT
FROM products_history
ORDER BY product_id, __START_AT;
```

- Spoon: two versions. Seattle (closed, `__END_AT` set) and Los Angeles (current, `__END_AT = NULL`).
- Fork: two versions. Cutlery category (closed) and Dining category (current).
- Napkin and Coaster: one version each (newly inserted, `__END_AT = NULL`).
- All other products: one version each (`__END_AT = NULL`).

 Ask Genie

Step 5: Process deletes through the pipeline

Now simulate two discontinued products by deleting them from the source table:

SQL

```
DELETE FROM products WHERE product_id = 9;  
DELETE FROM products WHERE product_id = 10;
```

These delete events are recorded in the CDF log but the streaming table has not seen them yet. Refresh the streaming table to process the new events:

SQL

```
REFRESH STREAMING TABLE products_history;
```

Query the dimension table to verify the deletes were applied:

SQL

```
SELECT product_id, product_name, warehouse, __START_AT, __END_AT  
FROM products_history  
ORDER BY product_id, __START_AT;
```

Bowl and Glass are now closed out with `__END_AT` set, marking them as discontinued. All other current products remain unchanged. The streaming table only processed the new delete events without reprocessing the inserts and updates from the previous refresh.

Step 6: Create an aggregate materialized view

Now that you have a dimension table that stays current with source changes, you can add a reporting layer on top.

A [materialized view](#) stores pre-computed query results as a physical table. Unlike a regular view, which re-executes the query each time you read from it, a materialized view persists the results and only recomputes the rows affected by upstream changes on each refresh. This makes it well-suited for dashboards and reports where query performance matters.

SQL

```
CREATE OR REPLACE MATERIALIZED VIEW products_by_category
SCHEDULE REFRESH EVERY 1 DAY
AS
SELECT
    category,
    COUNT(*) AS active_products
FROM products_history
WHERE __END_AT IS NULL
GROUP BY category;
```

`SCHEDULE REFRESH EVERY 1 DAY` means this view refreshes on a daily schedule.

Combined with the same schedule on the streaming table, you now have a three-stage pipeline where changes to the source table cascade through the dimension and into the aggregate on each refresh cycle. There is no manual refresh to run.

SQL

```
SELECT * FROM products_by_category ORDER BY active_products DESC;
```

Step 7: Verify the end-to-end cascade

To verify the full pipeline cascade, make a change to the source table:

SQL

```
UPDATE products SET warehouse = 'Seattle' WHERE product_id = 3;
```

The Knife moves from Denver to Seattle. This single DML change triggers the full pipeline cascade, demonstrating how the three stages work together:

1. `products` records the change event via CDF.
2. `products_history` processes the event and adds a new version for the Knife.
3. `products_by_category` recomputes only the affected Cutlery row.

Verify:

SQL

```
SELECT product_id, product_name, warehouse, __START_AT, __END_AT
FROM products_history
WHERE product_id = 3
ORDER BY __START_AT;

SELECT * FROM products_by_category ORDER BY active_products DESC;
```

Clean up

To clean up the resources created by this tutorial, use the following SQL:

SQL

```
DROP MATERIALIZED VIEW IF EXISTS products_by_category;
DROP STREAMING TABLE IF EXISTS products_history;
DROP TABLE IF EXISTS products;
```

Additional resources

- [Streaming tables](#)
- [Materialized views](#)
- [CREATE STREAMING TABLE ... FLOW AUTO CDC](#)
- [The AUTO CDC APIs: Simplify change data capture with pipelines](#)
- [Lakeflow Spark Declarative Pipelines](#)

Is this page

as this page helpful?

☐ Yes	☐ No
☐ Send feedback	